

M12x — Cell-by-cell logical inspection (18 cells)

Experiment ID: M12x (Milestone 1 Part 2 expanded)

Status: Stage-3 qualitative inspection **in progress** (1/18 written)

Date started: 2026-07-20

Parent record: M1.2-expanded.md

Artefacts: causal/outputs/aba_learning/grid/M12x_{ecai2024,aamas2025}/cells/{dgp}__target-{t}/

Side-car matrix: causal/outputs/aba_learning/grid/M12x_summary.md

Per-cell detector reports: causal/outputs/aba_learning/grid/M12x_cell_reports/

Fixtures: causal/experiments/handcrafted_m12x.py

Catalogue: docs/research/milestone_plans/milestone1_part2/m12x_units_reference.tex

Grid run: 2026-07-20 (18/18 cells solved; summary regenerated same day)

This document inspects each of the 18 (**arm** $\times$ **fixture** $\times$ **target**) cells. For every completed cell it records: the locked setup and $\mathcal{H}_t^*$; a clear reading of how ABA Learning produced the learned framework; and whether and why that framework matches or fails the intended hypothesis and semantic success description.

This is **not** Russo-style Causal ABA. The object of study is unguided ABA Learning recovery of mechanism-aligned target rules from categorical tabular background knowledge and positive/negative examples.

Reading conventions. Learned δ is the new intensional material in `bk.sol.aba` after variable-canonical background knowledge subtraction. Exact match means the learned target rules equal $\mathcal{H}_t^*$ after normalisation. ASP `pos_covered` / `neg_rejected` fractions are coverage facts from the side-car; they are not a substitute for intensional match. Acceptance remains inspection-first against the Approach semantic description.

1 Inspection checklist (18)

#	Arm	Fixture	Target	Exact $\mathcal{H}^*$?	Short verdict
1	ECAI	m12_u1_separator_copy	x_2	N	Assumption-mediated via isolated x_0 ; parent only in contraries
2	ECAI	m12_u2_collider_min	x_2	N	Elegant assumption-mediated both-parents-nonzero (not the locked 4-clause form)
3	ECAI	m12_u3_collider_max	x_2	—	<i>pending</i>
4	ECAI	m12_u4_fork_asymmetric	x_1	—	<i>pending</i>
5	ECAI	m12_u4_fork_asymmetric	x_2	—	<i>pending</i>
6	ECAI	m12_u5_chain_curated	x_2	—	<i>pending</i>
7	ECAI	m12_u6_g1_min_diff	x_2	—	<i>pending</i>
8	ECAI	m12_u6_g1_min_diff	x_3	—	<i>pending</i>
9	ECAI	m12_u7_diamond_noisy	x_3	—	<i>pending</i>
10	AAMAS	m12_u1_separator_copy	x_2	N	Six specialised conjunctions; isolated x_0 retained beside parent

#	Arm	Fixture	Target	Exact $\mathcal{H}^*$?	Short verdict
11	AAMAS	m12_u2_collider_min	x_2	—	<i>pending</i>
12	AAMAS	m12_u3_collider_max	x_2	—	<i>pending</i>
13	AAMAS	m12_u4_fork_asymmetric	x_1	—	<i>pending</i>
14	AAMAS	m12_u4_fork_asymmetric	x_2	—	<i>pending</i>
15	AAMAS	m12_u5_chain_curated	x_2	—	<i>pending</i>
16	AAMAS	m12_u6_g1_min_diff	x_2	—	<i>pending</i>
17	AAMAS	m12_u6_g1_min_diff	x_3	—	<i>pending</i>
18	AAMAS	m12_u7_diamond_noisy	x_3	—	<i>pending</i>

2 Shared regime (locked)

- Alphabet $K = \{0, 1, 2\}$; nonzero-positive labels $E^+ = \{t \neq 0\}$, $E^- = \{t = 0\}$ unless a unit card overrides.
- Background knowledge: exact-value predicates `xi_val_v` for every non-target column (descendants kept as distractors).
- Configs: ECAI (`configs/ecai2024_config.pl`); AAMAS (`configs/aamas2025_config.pl`).
- Reference hypotheses $\mathcal{H}_t^*$ and structural sets are as in the unit catalogue and `m12x_summary.py` look-up tables.

Effective options (confirmed in cell traces):

- **ECAI:** brave learning; `folding_mode(nd)`; `folding_selection(any)`; `folding_space(all)`; `asm_intro(relto)`; `post_folding_test_ entailment(true)`.
- **AAMAS:** brave learning; greedy folding; published AAMAS options consulted verbatim from `configs/aamas2025_config.pl`.

3 Batch A — ECAI (M12x_ecai2024)

3.1 1. ECAI $\times$ m12_u1_separator_copy $\times$ $t = x_2$

Cell directory: M12x_ecai2024/cells/m12_u1_separator_copy__target-x2/

Runner outcome: solved

ASP coverage: positives 6/6, negatives 3/3

3.1.1 Setup

The graph is a separator $x_1 \rightarrow x_2$ with an isolated variable x_0 . The mechanism sets x_2 equal to x_1 on the full nine-row factorial over (x_0, x_1) . Under nonzero-positive labelling the positive examples are rows $\{2, 3, 5, 6, 8, 9\}$ and the negative examples are rows $\{1, 4, 7\}$. Background knowledge contains exact-value predicates for x_0 and x_1 ; the target column x_2 is excluded. The ancestors of x_2 are $\{x_1\}$; the isolated variable x_0 is a distractor; there are no descendants. The design role of this unit is to test whether the learner recovers a unary parent rule and avoids citing the isolated distractor.

3.1.2 Intended hypothesis $\mathcal{H}_{x_2}^*$

```
x2(A) :- x1_val_1(A).
x2(A) :- x1_val_2(A).
```

These two clauses characterise $x_2 \neq 0$ by the parent x_1 being nonzero. They must not cite the isolated distractor x_0 .

3.1.3 What happened

This cell uses the ECAI published configuration on unit U1 with learning target x_2 . The learner returned a solved framework with full ASP coverage of the examples. That framework does not match $\mathcal{H}_{x_2}^*$ intensionally. The target rules cite the isolated distractor x_0 together with assumptions, and the true parent x_1 appears only in the contrary conditions.

3.1.4 What the learner did

Learning begins from rote rules that cover each positive example by its sample identifier:

```
x2(A) :- A=2.
x2(A) :- A=3.
x2(A) :- A=5.
x2(A) :- A=6.
x2(A) :- A=8.
x2(A) :- A=9.
```

Under non-deterministic folding with selection **any** and folding space **all**, the first successful fold for a representative positive in each x_0 -slice replaces that identifier by a value predicate of the isolated column x_0 . The first fold yields a body containing only `x0_val_0`; later folds yield only `x0_val_1` and only `x0_val_2`:

```
x2(A) :- x0_val_0(A).    % from A=2; overgeneralises within the x0=0 slice
x2(A) :- x0_val_1(A).    % from A=5
x2(A) :- x0_val_2(A).    % from A=8
```

Each of those bodies overgeneralises within its slice, because the negative example that shares the same x_0 value would also be covered. Entailment of the labelled example set therefore fails, and assumption introduction relative to the failing body adds a fresh assumption `alpha_1`, `alpha_2`, or `alpha_3`. The remaining positive examples in the same slice are then deleted as subsumed. The repaired target rules are:

```
x2(A) :- alpha_1(A), x0_val_0(A).
x2(A) :- alpha_2(A), x0_val_1(A).
x2(A) :- alpha_3(A), x0_val_2(A).
```

Contrary learning is driven by the three negative examples, which are exactly the rows with $x_1 = 0$. For each contrary, folding first tries the matching x_0 literal already used in the corresponding target rule. That attempt is rejected, because the engine cannot introduce an assumption already tied to that body. Folding then succeeds with `x1_val_0`:

```
c_alpha_1(A) :- x1_val_0(A).
c_alpha_2(A) :- x1_val_0(A).
c_alpha_3(A) :- x1_val_0(A).
```

The accepted solution therefore has three assumption-mediated target rules keyed by x_0 , and three identical contrary rules that block those assumptions whenever $x_1 = 0$. Body-scope variables of the target rules are $\{x_0\}$. The parent x_1 appears only in the framework-scope contrary conditions.

3.1.5 Match assessment

The learned target rules are not an exact match to $\mathcal{H}_{x_2}^*$.

On the complete partition of the table by x_0 , an assumption remains undefeated on a row in that slice if and only if `x1_val_0` fails, which is exactly when $x_1 \in \{1, 2\}$. Taken together, the three rules therefore cover the six positives and reject the three negatives. In that limited extensional sense the solution agrees with the nonzero-positive copy semantics of U1.

Intensionally the solution is misaligned with the intended hypothesis. The learner folded the isolated distractor x_0 before the true parent x_1 . It repaired the resulting overgeneralisation by introducing assumptions. It recovered the true separator only as contrary conditions on x_1 . The intended hypothesis cites x_1 directly in ordinary target rules and does not mention x_0 at all.

Short verdict. The cell is solved and extensionally adequate, but intensionally it encodes the parent concept through assumptions keyed by the isolated distractor, with the true parent appearing only in the contraries.

3.2 2. ECAI $\times$ m12_u2_collider_min $\times$ $t = x_2$

Cell directory: M12x_ecai2024/cells/m12_u2_collider_min__target-x2/

Runner outcome: solved

ASP coverage: positives 4/4, negatives 5/5

3.2.1 Setup

Unit U2 is a collider $x_0 \rightarrow x_2 \leftarrow x_1$ with mechanism $x_2 := \min(x_0, x_1)$ on the full nine-row factorial over (x_0, x_1) . Under nonzero-positive labelling, $x_2 \neq 0$ if and only if both parents are nonzero. The positive examples are therefore rows $\{5, 6, 8, 9\}$ and the negative examples are rows $\{1, 2, 3, 4, 7\}$. Background knowledge contains exact-value predicates for both parents; the target x_2 is excluded. There are no sibling or descendant distractors beyond the two parents themselves. The design role of this unit is to test recovery of a conjunctive both-parent rule. The locked reference $\mathcal{H}_{x_2}^*$ enumerates that concept as four ordinary `val` conjunctions, one for each nonzero pair of parent values.

3.2.2 Intended hypothesis $\mathcal{H}_{x_2}^*$

```
x2(A) :- x0_val_1(A), x1_val_1(A).
x2(A) :- x0_val_1(A), x1_val_2(A).
x2(A) :- x0_val_2(A), x1_val_1(A).
x2(A) :- x0_val_2(A), x1_val_2(A).
```

3.2.3 What happened

This cell uses the ECAI published configuration on unit U2 with learning target x_2 . The learner returned a solved framework with full ASP coverage. It did not emit the four locked case clauses. Instead it recovered a compact assumption-mediated representation of the same semantic idea: x_0 is nonzero in the target bodies, and x_1 is nonzero through undefeated assumptions whose contraries fire exactly when $x_1 = 0$.

3.2.4 What the learner did

Learning begins from rote rules for the four positives:

```
x2(A) :- A=5.  
x2(A) :- A=6.  
x2(A) :- A=8.  
x2(A) :- A=9.
```

Under non-deterministic folding with selection **any** and folding space **all**, the first fold for $A = 5$ (where $x_0 = 1$) replaces the sample identifier by `x0_val_1` alone:

```
x2(A) :- x0_val_1(A).
```

That body overgeneralises: it would also cover the negative row $A = 4$, where $x_0 = 1$ but $x_1 = 0$. Entailment of the labelled example set therefore fails, and assumption introduction adds `alpha_1`:

```
x2(A) :- alpha_1(A), x0_val_1(A).
```

The remaining positive in the same $x_0 = 1$ slice ($A = 6$) is then deleted as subsumed. The same pattern occurs for the $x_0 = 2$ slice: fold $A = 8$ to `x0_val_2`, introduce `alpha_2`, and delete $A = 9$ as subsumed:

```
x2(A) :- alpha_2(A), x0_val_2(A).
```

Contrary learning is driven by the negatives that sit in those same x_0 slices, namely $A = 4$ and $A = 7$ (both have $x_1 = 0$). Folding each contrary first tries the matching x_0 literal already used in the corresponding target rule; that attempt is rejected. Folding then succeeds with `x1_val_0`:

```
c_alpha_1(A) :- x1_val_0(A).  
c_alpha_2(A) :- x1_val_0(A).
```

In the ASP solution these assumptions are realised by default-negation rules scoped to the same x_0 -slices:

```
alpha_1(A) :- not c_alpha_1(A), x0_val_1(A).  
alpha_2(A) :- not c_alpha_2(A), x0_val_2(A).
```

Those clauses are what allow each assumption to hold on its slice precisely when the corresponding contrary fails, that is when $x_1 \neq 0$. Without them, `alpha_1` and `alpha_2` would never be derived, and the target rules that depend on them would not cover the positives.

Negatives with $x_0 = 0$ (rows $\{1, 2, 3\}$) never enter these target bodies, so they need no contrary repair. Body-scope variables of the target rules are $\{x_0\}$. The second parent x_1 appears in the contrary conditions, and thereby controls whether the assumptions succeed.

3.2.5 Match assessment

Relative to the locked case enumeration $\mathcal{H}_{x_2}^*$, this is not an exact syntactic match. The learner does not produce the four ordinary both-parent conjunctions.

Judged against the semantic success description for U2, however, the learned framework is a strong and arguably more elegant intensional solution. It characterises nonzero x_2 under min by combining an explicit nonzero condition on one parent in the target bodies with an assumption-mediated nonzero condition on the other parent. On an $x_0 \in \{1, 2\}$ row, x_2 holds if and only if the assumption is undefeated if and only if $x_1 \neq 0$. That is exactly “both parents are nonzero,” expressed with two target rules and the ABA assumption machinery rather than with four case clauses. ASP coverage is complete for that reason.

So the mismatch with $\mathcal{H}_{x_2}^*$ should not be read as a failure to recover the intended concept. It is a mismatch with one particular syntactic presentation of that concept. The learned form uses assumptions to represent nonzeroness compactly.

Short verdict. Not an exact match to the locked four-clause $\mathcal{H}_{x_2}^*$, but an elegant assumption-mediated encoding of the same both-parents-nonzero concept, with full coverage.

3.3 3. ECAI $\times$ m12_u3_collider_max $\times$ $t = x_2$

Cell directory: M12x_ecai2024/cells/m12_u3_collider_max__target-x2/

Runner outcome: solved

ASP coverage: positives 8/8, negatives 1/1

3.3.1 Setup

Unit U3 is the disjunctive counterpart of U2: the same collider $x_0 \rightarrow x_2 \leftarrow x_1$, now with mechanism $x_2 := \max(x_0, x_1)$ on the nine-row factorial. Under nonzero-positive labelling, $x_2 \neq 0$ if and only if at least one parent is nonzero. The positive examples are therefore rows $\{2, 3, 4, 5, 6, 7, 8, 9\}$ and the sole negative is row $\{1\}$. Background knowledge contains exact-value predicates for both parents; the target x_2 is excluded. The locked reference $\mathcal{H}_{x_2}^*$ is the four-clause `val` expansion of “at least one parent is nonzero.”

3.3.2 Intended hypothesis $\mathcal{H}_{x_2}^*$

```
x2(A) :- x0_val_1(A).
x2(A) :- x0_val_2(A).
x2(A) :- x1_val_1(A).
x2(A) :- x1_val_2(A).
```

3.3.3 What happened

This cell uses the ECAI published configuration on unit U3 with learning target x_2 . After the initial optimum there are eight rote rules, one for each positive. Non-deterministic folding proceeds by x_0 -slices and yields a mixed solution: pure unary rules for $x_0 \in \{1, 2\}$, and an assumption-mediated rule only on the incomplete $x_0 = 0$ slice. The learner returns a solved framework with full ASP coverage. The result is not an exact syntactic match to $\mathcal{H}_{x_2}^*$, but it is a compact case analysis that still expresses the intended disjunctive concept.

3.3.4 What the learner did

Learning begins from rote coverage of the eight positives. The first fold chosen is $A = 2$ (where $x_0 = 0, x_1 = 1$), which replaces the identifier by `x0_val_0` alone:

```
x2(A) :- x0_val_0(A).
```

That body would also cover the unique negative $A = 1$ (where $x_0 = 0$ and $x_1 = 0$), so entailment fails and assumption introduction adds `alpha_1`:

```
x2(A) :- alpha_1(A), x0_val_0(A).
```

The other positive in the same slice ($A = 3$) is then deleted as subsumed. Next, $A = 4$ folds to `x0_val_1`. Under max, every row with $x_0 = 1$ is already positive, so this one-literal rule entails $\langle E^+, E^- \rangle$ immediately and needs no assumption:

```
x2(A) :- x0_val_1(A).
```

Positives $A = 5$ and $A = 6$ are subsumed. Likewise $A = 7$ folds to a safe unary rule on the $x_0 = 2$ slice:

```
x2(A) :- x0_val_2(A).
```

Positives $A = 8$ and $A = 9$ are subsumed. Contrary learning for the negative $A = 1$ first tries `x0_val_0` (rejected), then succeeds with `x1_val_0`:

```
c_alpha_1(A) :- x1_val_0(A).
```

In the ASP solution the assumption is realised by

```
alpha_1(A) :- not c_alpha_1(A), x0_val_0(A).
```

so on the $x_0 = 0$ slice the assumption holds precisely when $x_1 \neq 0$. Body-scope variables of the target rules are $\{x_0\}$; x_1 appears only through the contrary / assumption pair.

3.3.5 Match assessment

This is not an exact match to $\mathcal{H}_{x_2}^*$. The learner does not emit the four unary parent rules of the locked OR expansion.

Extensionally, the learned theory is

$$(x_0 = 0 \wedge x_1 \neq 0) \vee (x_0 = 1) \vee (x_0 = 2),$$

which is exactly $x_0 \neq 0$ or $x_1 \neq 0$, the nonzero-positive semantics of max. ASP coverage is complete for that reason.

Intensionally this is an x_0 -first case analysis: the complete nonzero x_0 slices become ordinary unary rules, and only the incomplete $x_0 = 0$ slice needs an assumption to encode “ x_1 nonzero.” Relative to $\mathcal{H}_{x_2}^*$, the presentation differs (no free-standing `x1_val_1` / `x1_val_2` target rules), but the recovered concept is the intended disjunction. The pattern is related to ECAI on U2: assumption machinery is used where a one-literal fold would overgeneralise, while folds that are already safe are kept without assumptions.

Short verdict. Not an exact match to the locked four-rule OR, but a solved, fully covering x_0 -case analysis that encodes the same “at least one parent nonzero” concept, with assumptions only on the $x_0 = 0$ slice.

4 Batch B — AAMAS (M12x_aamas2025)

4.1 10. AAMAS $\times$ m12_u1_separator_copy $\times$ $t = x_2$

Cell directory: M12x_aamas2025/cells/m12_u1_separator_copy__target-x2/

Runner outcome: solved

ASP coverage: positives 6/6, negatives 3/3

4.1.1 Setup

Same unit and target as cell 1: separator $x_1 \rightarrow x_2$ with isolated x_0 ; $x_2 := \text{copy}(x_1)$ on the nine-row factorial; nonzero-positive examples $E^+ = \{2, 3, 5, 6, 8, 9\}$ and $E^- = \{1, 4, 7\}$; exact-value background knowledge for x_0 and x_1 . This arm uses the AAMAS published configuration (`folding_mode(greedy)`, `folding_selection(mgr)`, `folding_space(bk)`, `asm_intro(relto)`).

4.1.2 Intended hypothesis $\mathcal{H}_{x_2}^*$

```
x2(A) :- x1_val_1(A).
x2(A) :- x1_val_2(A).
```

4.1.3 What happened

This cell uses the AAMAS published configuration on the same U1 instance. The learner returned a solved framework with full ASP coverage and with no assumptions or contraries. The learned target theory is a six-rule case split: each positive row becomes a conjunction of that row’s x_0 value and x_1 value. The parent x_1 appears in every body, but so does the isolated distractor x_0 . The result is therefore not an exact match to $\mathcal{H}_{x_2}^*$.

4.1.4 What the learner did

Learning again begins from rote rules for the six positives. Under greedy folding with selection `mgr` and folding space restricted to background knowledge, each positive is folded by adjoining both column values present on that row. For example, sample $A = 2$ has $x_0 = 0$ and $x_1 = 1$, and folds to:

```
x2(A) :- x0_val_0(A), x1_val_1(A).
```

Each such two-literal body entails the labelled example set immediately, so no assumption introduction is triggered. The same pattern is applied independently to every positive. The accepted target rules are:

```
x2(A) :- x0_val_0(A), x1_val_1(A).
x2(A) :- x0_val_0(A), x1_val_2(A).
x2(A) :- x0_val_1(A), x1_val_1(A).
```



```
x2(A) :- x0_val_1(A), x1_val_2(A).
x2(A) :- x0_val_2(A), x1_val_1(A).
x2(A) :- x0_val_2(A), x1_val_2(A).
```

There is no further generalisation that drops the x_0 literals and merges the six clauses into the two unary parent rules of $\mathcal{H}_{x_2}^*$. Body-scope variables are $\{x_0, x_1\}$. The solution uses no assumptions and no contraries.

4.1.5 Match assessment

The learned target rules are not an exact match to $\mathcal{H}_{x_2}^*$.

Extensionally, under mutual exclusivity and exhaustiveness of the three x_0 value predicates, the six conjunctions cover exactly the rows where $x_1 \in \{1, 2\}$. That is the same labelling as the intended two-rule parent hypothesis, which is why ASP coverage is complete.

Intensionally the solution is a positive-row case split that retains the isolated distractor in every body. The true parent is present, but the theory is a parent-plus-distractor superset of $\mathcal{H}_{x_2}^*$, not the compact unary parent rules. Relative to the ECAI twin on this unit, AAMAS avoids assumption-mediated repair and keeps x_1 in the target bodies, but it still fails to eliminate x_0 .

Short verdict. The cell is solved and extensionally adequate, with no assumptions, but intensionally it enumerates six specialised conjunctions that keep the isolated distractor beside the true parent.

4.2 11. AAMAS $\times$ m12_u2_collider_min $\times$ $t = x_2$

Cell directory: M12x_aamas2025/cells/m12_u2_collider_min__target-x2/

Runner outcome: solved

ASP coverage: positives 4/4, negatives 5/5

4.2.1 Setup

Same unit and target as cell 2: collider $x_0 \rightarrow x_2 \leftarrow x_1$ with $x_2 := \min(x_0, x_1)$ on the nine-row factorial; nonzero-positive examples $E^+ = \{5, 6, 8, 9\}$ and $E^- = \{1, 2, 3, 4, 7\}$; exact-value background knowledge for both parents. This arm uses the AAMAS published configuration (`folding_mode(greedy)`, `folding_selection(mgr)`, `folding_space(bk)`, `asm_intro(relto)`).

4.2.2 Intended hypothesis $\mathcal{H}_{x_2}^*$

```
x2(A) :- x0_val_1(A), x1_val_1(A).
x2(A) :- x0_val_1(A), x1_val_2(A).
x2(A) :- x0_val_2(A), x1_val_1(A).
x2(A) :- x0_val_2(A), x1_val_2(A).
```

4.2.3 What happened

This cell uses the AAMAS published configuration on the same U2 instance. After the initial optimum, the engine has four rote rules covering the positives by sample identifier:

```

x2(A) :- A=5.
x2(A) :- A=6.
x2(A) :- A=8.
x2(A) :- A=9.

```

The trace then computes a greedy folding plan for each of those four rules. For every positive, greedy/**mgr** first replaces the identifier by the x_0 value on that row, then extends the body with the x_1 value on the same row. The planned foldings are therefore the four nonzero parent pairs:

- $A = 5 \rightarrow x0_val_1, x1_val_1$
- $A = 6 \rightarrow x0_val_1, x1_val_2$
- $A = 8 \rightarrow x0_val_2, x1_val_1$
- $A = 9 \rightarrow x0_val_2, x1_val_2$

Those foldings are then applied one at a time. After each application, entailment is checked against the **whole current theory**, not against the newly folded clause alone. So after the first fold, the theory still contains the remaining rote rules for $A = 6, 8, 9$; after the second, the remaining rote rules for $A = 8, 9$; and so on. Each intermediate theory already separates $E^\pm$, so the trace reports **extended ABA entails $\langle E^+, E^- \rangle$ - using folding** and never introduces assumptions. Learning stops with **nothing to fold**. The final solution is the four specialised conjunctions, with no assumptions and no contraries, and with full ASP coverage.

4.2.4 What the learner did

The accepted target rules are:

```

x2(A) :- x0_val_1(A), x1_val_1(A).    % from A=5
x2(A) :- x0_val_1(A), x1_val_2(A).    % from A=6
x2(A) :- x0_val_2(A), x1_val_1(A).    % from A=8
x2(A) :- x0_val_2(A), x1_val_2(A).    % from A=9

```

Body-scope variables are $\{x_0, x_1\}$.

4.2.5 Match assessment

This is an exact match to $\mathcal{H}_{x_2}^*$.

The four conjunctions are precisely the locked **val** expansion of “both parents are nonzero.” ASP coverage is complete, and the solution uses no assumptions.

Contrast with ECAI on the same unit. Under ECAI (cell 2), non-deterministic folding first produces one-literal bodies such as $x2(A) :- x0_val_1(A)$, which overgeneralise within an x_0 -slice and force assumption introduction; the second parent is recovered only through contraries, yielding a compact assumption-mediated encoding of both-parents-nonzero that is semantically successful but not an exact syntactic match to $\mathcal{H}_{x_2}^*$. Under AAMAS, greedy folding specialises each positive to the full parent pair immediately, so each intermediate theory remains safe without assumptions and the final rules coincide with $\mathcal{H}_{x_2}^*$. Both arms recover the intended concept; they differ in presentation: ECAI’s assumption form versus AAMAS’s exact case enumeration.

Short verdict. Exact match to $\mathcal{H}_{x_2}^*$: four both-parent conjunctions, no assumptions, full coverage.

Cells 3–9 and 12–18 to follow.